Truck Asset Matchmaking Platform (TAMP)

Architecture & Design

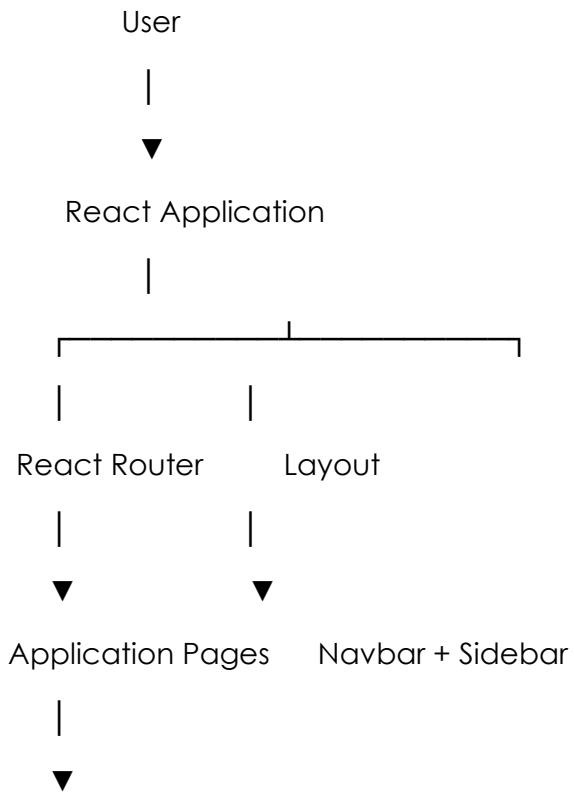
1. Architecture Overview

The Truck Asset Matchmaking Platform (TAMP) follows a component-based frontend architecture built using React.

The application is organised into reusable pages and components that communicate through React props, local state, and mock data. The architecture separates layout, navigation, business pages, and reusable UI components to improve maintainability and scalability.

The MVP uses mock data instead of a backend API, allowing the complete business workflow to be demonstrated without server-side implementation.

2. High-Level Architecture



Reusable Components

|

▼

Mock Data / Local State

3. Application Structure

The application is divided into the following modules:

Authentication

- Login
- Register
- Role Selection

Freight Owner

- Dashboard
- Post Load
- My Loads
- Match Recommendations
- Digital Confirmation
- Trip Tracking
- Ratings

Transporter

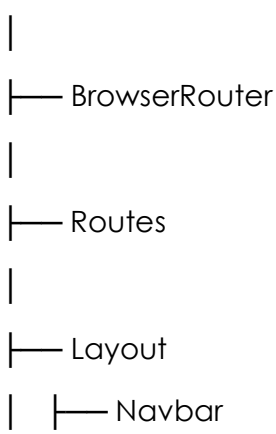
- Dashboard
- Post Truck
- My Trucks
- Available Loads

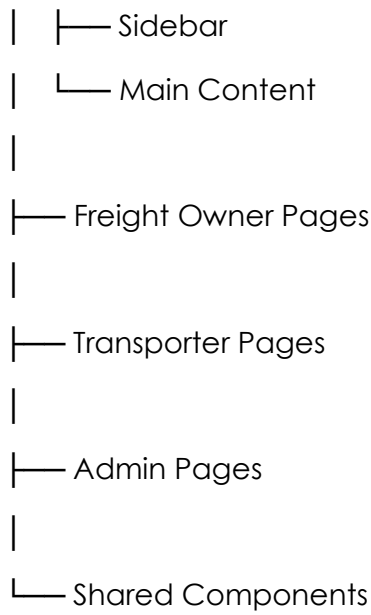
- Trip Tracking
- Ratings

Administrator
- Dashboard
- Users
- Compliance
- Audit Trail
- Analytics

Shared Components
- Layout
- Navbar
- Sidebar
- Dashboard Cards
- Forms
- Buttons

Component Hierarchy

App




5. Routing Design

React Router DOM is used for client-side routing.

Example routes include:

- /
- /login
- /role-selection

Freight Owner

- /freight-owner/dashboard
- /freight-owner/post-load
- /freight-owner/my-loads
- /freight-owner/matches

Transporter

- /transporter/dashboard

- /transporter/post-truck
- /transporter/my-trucks

Administrator
- /admin/dashboard
- /admin/users
- /admin/compliance
- /admin/audit
- /admin/analytics

Shared
- /tracking
- /confirmation

6. State Management

The MVP uses React's local state (useState) together with mock data.

State is primarily used for:
- Form inputs
- Navigation
- Match acceptance
- Dashboard information
- Tracking progress
- Ratings

7. Data Flow

The application follows a simple one-directional data flow.

Updated User Interface

8. Design Decisions

The following design decisions were made during development:

- Component-based architecture for reusability.
- Shared Layout component across all dashboards.
- Role-based navigation using a dynamic Sidebar.
- Tailwind CSS for rapid UI development.
- Mock data instead of backend services.
- Responsive layouts using CSS Grid and Flexbox.
- Reusable cards, buttons, and dashboard components.

9. Conclusion

The frontend architecture provides a modular, maintainable, and scalable foundation for the Truck Asset Matchmaking Platform. The separation of concerns between routing, layout, reusable components, and feature pages makes the application easy to extend while supporting the complete MVP workflow.